

Test Case: Inclusion-Maximal Remainder Sets

This test verifies the engine's ability to calculate the remainder set $A \perp \phi$. We use a Belief Base A where multiple distinct subsets can be formed that do not imply the target formula ϕ .

1. The Input Belief Base (Unsorted Set)

The following set of formulas is provided to the engine. The order of these formulas in the input list should not affect the outcome.

$$A = \{p, \quad q, \quad p \leftrightarrow q, \quad r\}$$

2. The Operation

We wish to contract the formula p :

$$A \div p$$

3. Logical Analysis (Manual Calculation)

The Belief Base A entails p in two distinct ways:

1. Directly: via the formula $\{p\}$.
2. Indirectly: via the combination of $\{q, p \leftrightarrow q\}$.

The formula $\{r\}$ is "irrelevant" to p and should be present in all remainders.

4. Expected Remainder Sets ($A \perp p$)

To find the remainders, your code must find every maximal subset of A that does not imply p . There are exactly two such sets:

- **Remainder 1 (S_1):** $\{q, r\}$
(Adding p or $p \leftrightarrow q$ would make p true).
- **Remainder 2 (S_2):** $\{p \leftrightarrow q, r\}$
(Adding p or q would make p true).

Result of $A \perp p$: $\{\{q, r\}, \{p \leftrightarrow q, r\}\}$

5. Expected Output for Expansion/Revision

If your code implements ****Partial Meet Contraction****, the output depends on your selection function γ :

- **Full Meet:** The intersection $S_1 \cap S_2 = \{r\}$.
- **Partial Meet:** Either $\{q, r\}$ or $\{p \leftrightarrow q, r\}$ (or their intersection).

6. Verification for the Report

To prove the code works on sets:

1. Does the code identify that $\{r\}$ must stay because it is not part of any minimal path to p ?
2. Does the code find both S_1 and S_2 ?
3. If you change the order of the input set to $\{r, p \leftrightarrow q, q, p\}$, does the output remains identical?

How to use this with your code: Input: Pass the set [Literal "p"; Literal "q"; BiImplication(Literal "p", Literal "q"); Literal "r"] to your contraction function. CNF Note: Your engine will likely convert $p \leftrightarrow q$ into $(\neg p \vee q) \wedge (p \vee \neg q)$. Ensure your remainder logic treats these as a single unit from the belief base or handles the split clauses correctly. Target: Run Contract "p". Success: The test is successful if the engine refuses to keep $p \vee p$ and also refuses to keep both $q \vee q$ and $p \vee q$ at the same time.